

Problem List

Submit

Premium

Description

Accepted

Editorial

Solutions

Submissions

1480. Running Sum of 1d Array

Solved

EasyTopicsCompaniesHint

Given an array `nums`. We define a running sum of an array as `runningSum[i] = sum(nums[0]..nums[i])`.

Return the running sum of `nums`.

Example 1:

Input: `nums = [1,2,3,4]`
Output: `[1,3,6,10]`
Explanation: Running sum is obtained as follows: `[1, 1+2, 1+2+3, 1+2+3+4]`.

Example 2:

Input: `nums = [1,1,1,1,1]`
Output: `[1,2,3,4,5]`
Explanation: Running sum is obtained as follows: `[1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1]`.

Example 3:

Input: `nums = [3,1,2,10,1]`
Output: `[3,4,6,16,17]`

Constraints:

- `1 <= nums.length <= 1000`
- `-10^6 <= nums[i] <= 10^6`

8.9K22154 Online

Code

JavaAuto

```
1 class Solution {
2     public int[] runningSum(int[] nums) {
3         for (int i = 1; i < nums.length; i++) {
4             nums[i] += nums[i - 1];
5         }
6         return nums;
7     }
8 }
```

SavedLn 8, Col 2

Testcase

Test Result

AcceptedRuntime: 0 ms

Case 1Case 2Case 3

Input

nums =
[1, 2, 3, 4]

Output

1120101

28°C
Partly sunny

Search

ENG
IN

09:18
05-08-2025

Problem List

Submit

0

Premium

Description

Editorial

Solutions

Submissions

1672. Richest Customer Wealth

Easy

Topics

Companies

Hint

You are given an $m \times n$ integer grid `accounts` where `accounts[i][j]` is the amount of money the i^{th} customer has in the j^{th} bank. Return the **wealth** that the richest customer has.

A customer's **wealth** is the amount of money they have in all their bank accounts. The richest customer is the customer that has the maximum **wealth**.

Example 1:

Input: `accounts = [[1,2,3],[3,2,1]]`

Output: 6

Explanation:

1st customer has wealth = 1 + 2 + 3 = 6

2nd customer has wealth = 3 + 2 + 1 = 6

Both customers are considered the richest with a wealth of 6 each, so return 6.

Example 2:

Input: `accounts = [[1,5],[7,3],[3,5]]`

Output: 10

Explanation:

1st customer has wealth = 6

2nd customer has wealth = 10

3rd customer has wealth = 8

The 2nd customer is the richest with a wealth of 10.

Example 3:

Input: `accounts = [[2,8,7],[7,1,3],[1,9,5]]`

Output: 17

Code

Java

Auto

```
1 class Solution {
2     public int maximumWealth(int[][] accounts) {
3         int maxWealth = 0;
4
5         for (int[] customer : accounts) {
6             int wealth = 0;
7             for (int money : customer) {
8                 wealth += money;
9             }
10            maxWealth = Math.max(maxWealth, wealth);
11        }
12
13        return maxWealth;
14    }
15 }
```

Saved

Ln 15, Col 2

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Input

accounts =
[[1,2,3],[3,2,1]]

Output

4.9K 110 30 Online

Problem List

Submit

0

Premium

Description

Editorial

Solutions

Submissions

977. Squares of a Sorted Array

Solved

Easy

Topics

Companies

Given an integer array `nums` sorted in **non-decreasing** order, return an array of *the squares of each number* sorted in **non-decreasing** order.

Example 1:

Input: `nums = [-4,-1,0,3,10]`
Output: `[0,1,9,16,100]`
Explanation: After squaring, the array becomes `[16,1,0,9,100]`. After sorting, it becomes `[0,1,9,16,100]`.

Example 2:

Input: `nums = [-7,-3,2,3,11]`
Output: `[4,9,9,49,121]`

Constraints:

- `1 <= nums.length <= 104`
- `-104 <= nums[i] <= 104`
- `nums` is sorted in **non-decreasing** order.

Follow up: Squaring each element and sorting the new array is very trivial, could you find an `O(n)` solution using a different approach?

10.4K 195 102 Online

Code

Java

Auto

```
1 class Solution {
2     public int[] sortedSquares(int[] nums) {
3         int n = nums.length;
4         int[] result = new int[n];
5
6         int left = 0;
7         int right = n - 1;
8         int index = n - 1;
9
10        while (left <= right) {
11            int leftSquare = nums[left] * nums[left];
12            int rightSquare = nums[right] * nums[right];
13
14            if (leftSquare > rightSquare) {
15                result[index] = leftSquare;
16                left++;
17            } else {
18                result[index] = rightSquare;
19                right--;
20            }
21        }
22    }
23 }
```

Saved

Ln 26, Col 2

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

nums =

[-4,-1,0,3,10]

Output

[0,1,9,16,100]

Problem List

724. Find Pivot Index

Description

Editorial

Solutions

Submissions

Easy

Topics

Companies

Hint

Given an array of integers `nums`, calculate the **pivot index** of this array.

The **pivot index** is the index where the sum of all the numbers **strictly** to the left of the index is equal to the sum of all the numbers **strictly** to the index's right.

If the index is on the left edge of the array, then the left sum is `0` because there are no elements to the left. This also applies to the right edge of the array.

Return the **leftmost pivot index**. If no such index exists, return `-1`.

Example 1:

Input: `nums = [1,7,3,6,5,6]`
Output: `3`
Explanation:
The pivot index is 3.
Left sum = `nums[0] + nums[1] + nums[2] = 1 + 7 + 3 = 11`
Right sum = `nums[4] + nums[5] = 5 + 6 = 11`

Example 2:

Input: `nums = [1,2,3]`
Output: `-1`
Explanation:
There is no index that satisfies the conditions in the problem statement.

Example 3:

Input: `nums = [2,1,-1]`

Code

Java

Auto

```
1 class Solution {
2     public int pivotIndex(int[] nums) {
3         int totalSum = 0;
4
5         // Calculate total sum
6         for (int num : nums) {
7             totalSum += num;
8         }
9
10        int leftSum = 0;
11
12        // Find pivot index
13        for (int i = 0; i < nums.length; i++) {
14            if (leftSum == totalSum - leftSum - nums[i]) {
15                return i;
16            }
17            leftSum += nums[i];
18        }
19    }
20 }
```

Saved

Ln 22, Col 3

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Input

`nums = [1,7,3,6,5,6]`

Output

9.5K

275

79 Online

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array. The `Player` class is provided in the editor below. It has two fields:

1. `name`: a string.
2. `score`: an integer.

Given an array of n `Player` objects, write a comparator that sorts them in order of decreasing score. If 2 or more players have the same score, sort those players alphabetically ascending by name. To do this, you must create a `Checker` class that implements the `Comparator` interface, then write an `int compare(Player a, Player b)` method implementing the `Comparator.compare(T o1, T o2)` method. In short, when sorting in ascending order, a comparator function returns -1 if $a < b$, 0 if $a = b$, and 1 if $a > b$.

Declare a `Checker` class that implements the comparator method as described. It should sort first descending by score, then ascending by name. The code stub reads the input, creates a list of `Player` objects, uses your method to sort the data, and prints it out properly.

Example

$n = 3$ `data = [[Smith, 20], [Jones, 15], [Jones, 20]]`

Sort the list as `datasorted = [[Jones, 20], [Smith, 20], [Jones, 15]]`. Sort

```
11 }
12
13 class Checker implements Comparator<Player> {
14     // complete this method
15     public int compare(Player a, Player b) {
16         if (a.score != b.score) {
17             return b.score - a.score;
18         } else {
19             return a.name.compareTo(b.name);
20         }
21     }
22 }
23
24 > ...
```

Line: 22 Col: 2

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

☒ Sample Test case 0


Problem

Submissions

Leaderboard

You are given a list of student information: ID, FirstName, and CGPA. Your task is to rearrange them according to their CGPA in decreasing order. If two student have the same CGPA, then arrange them according to their first name in alphabetical order. If those two students also have the same first name, then order them according to their ID. No two students have the same ID.

Hint: You can use comparators to sort a list of objects. See the [oracle docs](#) to learn about comparators.

Input Format

The first line of input contains an integer N , representing the total number of students. The next N lines contains a list of student information in the following structure:

ID Name CGPA

Constraints

- $2 \leq N \leq 1000$
- $0 \leq ID \leq 100000$
- $5 \leq |Name| \leq 30$

```
26
27 class Solution {
28     public static void main(String[] args) {
29         Scanner in = new Scanner(System.in);
30
31         int testCases = Integer.parseInt(in.nextLine());
32         List<Student> studentList = new ArrayList<>();
33
34         while (testCases > 0) {
35             int id = in.nextInt();
36             String fname = in.next();
37             double cgpa = in.nextDouble();
38
39             Student st = new Student(id, fname, cgpa);
40             studentList.add(st);
41
42             testCases--;
```

Line: 65 Col: 1

Upload Code as File

☐ Test against custom input

RunCode

SubmitCode

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Problem

Submissions

Leaderboard

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array.

The Player class is provided for you in your editor. It has 2 fields: a *name* String and a *score* Integer.

Given an array of *n* Player objects, write a comparator that sorts them in order of decreasing score; if 2 or more players have the same score, sort those players alphabetically by name. To do this, you must create a Checker class that implements the Comparator interface, then write an int compare(Player a, Player b) method implementing the `Comparator.compare(T o1, T o2)` method.

Input Format

Input from stdin is handled by the locked stub code in the Solution class.
The first line contains an integer, *n*, denoting the number of players.
Each of the *n* subsequent lines contains a player's *name* and *score*, respectively.

Constraints

- $0 \leq \text{score} \leq 1000$
- 2 players can have the same name.

```

import java.util.*;

class Checker implements Comparator<Player> {

    public int compare(Player a, Player b) {

        if (a.score < b.score)
            return 1;        // Higher score comes first

        if (a.score > b.score)
            return -1;

        // If scores are equal, sort by name alphabetically
        return a.name.compareTo(b.name);
    }
}

> class Player{...
```

Line: 4 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

This Java 8 challenge tests your knowledge of [Lambda expressions](#)!

Write the following methods that return a lambda expression performing a specified action:

1. PerformOperation isOdd(): The lambda expression must return *true* if a number is odd or *false* if it is even.
2. PerformOperation isPrime(): The lambda expression must return *true* if a number is prime or *false* if it is composite.
3. PerformOperation isPalindrome(): The lambda expression must return *true* if a number is a palindrome or *false* if it is not.

Input Format

Input is handled for you by the locked stub code in your editor.

Output Format

The locked stub code in your editor will print *T* lines of output.

Sample Input

The first line contains an integer, *T* (the number of test cases).

The *T* subsequent lines each describe a test case in the form of 2 space-separated integers:

The first integer specifies the condition to check for (1 for Odd/Even, 2 for

```
59
60     int T = sc.nextInt();
61
62     while (T-- > 0) {
63         int ch = sc.nextInt();
64         int num = sc.nextInt();
65
66         PerformOperation op;
67
68         if (ch == 1) {
69             op = ob.isOdd();
70             System.out.println(ob.checker(op, num) ? "ODD" : "EVEN");
71         } else if (ch == 2) {
72             op = ob.isPrime();
73             System.out.println(ob.checker(op, num) ? "PRIME" :
74                                     "COMPOSITE");
75         } else if (ch == 3) {
```

Line: 83 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.